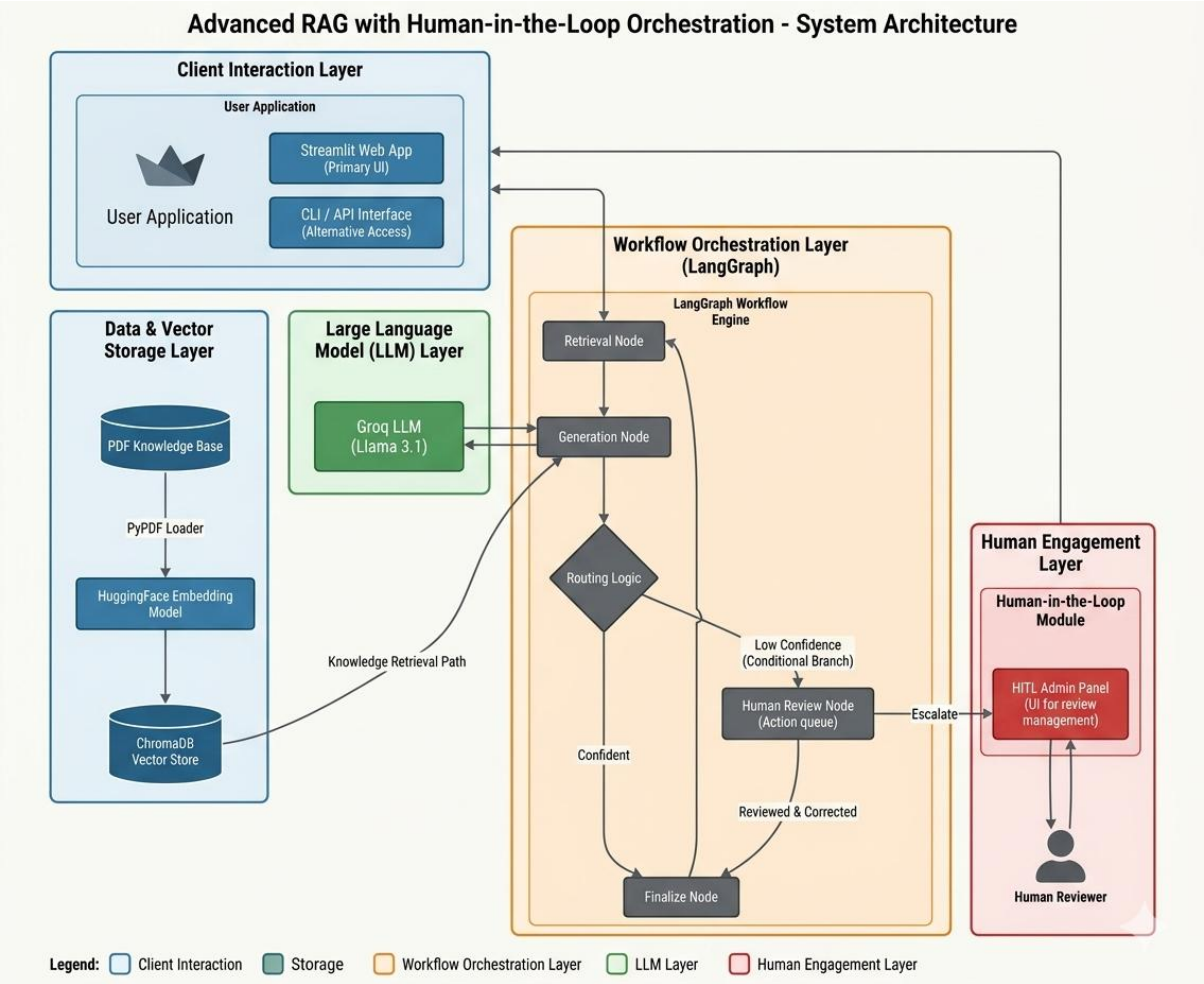
Technical Documentation

1. Introduction

What is RAG? Retrieval-Augmented Generation combines information retrieval with text generation. The system first retrieves relevant documents from a knowledge base, then feeds them as context to an LLM to produce a factual, grounded answer. Why It Is Needed: LLMs alone may hallucinate or lack proprietary data. RAG anchors responses in actual documents, making it ideal for customer support where accuracy is critical. Use Case Overview: A support assistant for an e-commerce company. Users ask policy questions; answers are drawn from an internal FAQ PDF. If the system is unsure, it escalates to a human agent.

2. System Architecture Explanation



The system is composed of four major layers:

- User Interface: Streamlit web app with chat panel and admin HITL panel.
- Orchestration Layer: LangGraph manages the sequential steps and conditional logic.
- Data Layer: ChromaDB stores embeddings; HuggingFace model generates embeddings; Groq LLM provides generation.
- Human-in-the-Loop: Streamlit UI component pauses and resumes the graph.
-

3. Design Decisions

- Chunk Size Choice: 500 characters with 100 overlap balances context completeness with retrieval granularity.
- Embedding Strategy: all-MiniLM-L6-v2 offers a good speed/quality trade-off, runs locally without API costs.
- Retrieval Approach: Similarity search with L2 distance. Top 4 chunks chosen to provide sufficient context without overwhelming prompt.
- Prompt Design: Explicit instruction to answer only from context, reducing hallucinations. Chat history included for conversational continuity.

4. Workflow Explanation

LangGraph Usage: The state machine is defined with four main nodes (retrieve, generate, human_review, finalize). The state object RAGState carries all data between nodes.
Node Responsibilities:

- retrieve: Performs semantic search and confidence estimation.
- generate: Constructs prompt and calls LLM.
- human_review: Dummy node for pause point.
- finalize: Extracts final_answer.

State Transitions: Detailed in LLD section.

5. Conditional Logic

Intent Detection: Currently implicit via confidence; questions with high semantic similarity to training data get high confidence.
Routing Decisions: The should_escalate function checks needs_human boolean. The boolean is set in retrieve_node based on confidence score.

6. HITL Implementation

Role of Human Intervention: Ensures that low-confidence or missing-info queries can be manually corrected, improving answer quality and user trust.
Benefits: Prevents misinformation, builds a feedback loop for future improvements.
Limitations: Adds latency; requires a human operator available.

7. Challenges & Trade-offs

- Retrieval Accuracy vs Speed: Larger k increases recall but slows retrieval and LLM prompt size. We chose k=4.
- Chunk Size vs Context Quality: Smaller chunks improve retrieval precision but may lose context; overlap compensates.
- Cost vs Performance: Local embeddings are free but slower; Groq LLM is free tier but limited to certain models.

8. Testing Strategy

Testing Approach: Manual testing with a set of sample queries covering in-domain, out-of-domain, and edge cases.
Sample Queries:

Query	Expected Outcome
"What is your return policy?"	Direct answer from FAQ, high confidence.
"How can I contact CEO?"	No context → escalates to HITL.
"Tell me a joke"	Out-of-scope → low confidence, HITL.
Follow-up: "What about international?"	Memory recalls previous topic.

9. Future Enhancements

- Multi-document support: Extend ingestion to multiple PDFs or web pages.
- Feedback loop: Use approval/reject data to fine-tune retrieval or LLM.
- Memory integration: Persistent conversation memory across sessions.
- Deployment: Containerize with Docker and deploy to Streamlit Cloud or AWS.